

Class 16: Modeling, Experiment Tracking, Evaluation, and Decision-Making

Data-driven Systems Engineering

Agenda

- Modeling as abstraction and representation choice.
- Matching model families to problem types and constraints.
- Why experimentation is engineering practice, not casual score-hunting.
- What must be tracked to compare models fairly.
- Tuning strategies and Neptune-style experiment logging in this repository.
- From a tuned run to a governed, traceable artifact.
- Evaluation must match the task, not the other way around.
- From a prediction to a decision: what interfaces must communicate.
- Thresholds, uncertainty, and operational cost.
- Where trust in a model interface breaks down.

Learning goals

Learning goals

- Frame modeling as an engineering choice among accuracy, interpretability, speed, and maintainability.
- Match model families to data, objectives, and operational constraints.
- Design an experiment before running it, and know what must be logged to compare runs fairly.
- Explain how hyperparameter search connects to reproducibility and later delivery.
- Choose evaluation metrics that reflect the real decision, not a generic leaderboard score.
- Distinguish a prediction from a decision, and identify what a decision-support interface must expose.
- Recognize the practical failure modes that erode trust in a served model.

Course Map and Running Cases



Today in the course

Focus: Choosing model families, tracking experiments honestly, and turning the winning run into a trustworthy decision.

Bridge: With features settled, this class asks how to compare modeling choices, keep evidence for why one model was selected, and then judge whether that model is actually good for the decision it supports.

Fraud case

In the fraud case, we compare model families and hyperparameters for a rare-event classifier, log every run so results stay reproducible, and then ask what threshold and interface turn a fraud score into a safe

Pasteurization case

In the pasteurization case, experiment tracking compares forecasting variants without losing context between runs, and a dashboard must communicate sensor state and model output together so an

What is modeling?

- Modeling represents patterns in data so the system can generalize to unseen cases.
- Every model encodes assumptions about structure, noise, and decision boundaries.
- Choosing a model means choosing a trade-off among accuracy, interpretability, speed, and maintainability.

Useful modeling questions

1. What decision or prediction are we supporting?
2. What kind of target do we have: class, score, or continuous value?
3. What kinds of errors are expensive?
4. How fast must training and inference be?
5. How often do we expect the data distribution to change?

Linear models	Fast, simple, and often strong baselines.
Tree-based models	Good for heterogeneous tabular data and nonlinearity.
Probabilistic models	Useful when uncertainty and conditional structure matter.
Neural networks	Powerful for large unstructured or highly nonlinear problems.
Online models	Update incrementally as new observations arrive.

More realistic examples

- Fraud detection: random forest or gradient boosting with threshold tuning.
- Wine-quality-style regression: gradient boosting tuned by grid search and tracked in Neptune.
- Pasteurization forecasting: incremental regression, revisited fully in Class 18.
- Industrial anomaly detection: useful only if alert cost and false-positive rate are understood upfront.

Repository example: a baseline training pipeline

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42, stratify=y  
)  
  
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

- From 'streamlit/modeling.py', mirrored in 'notebooks/Class6_Model_Training_.ipynb'.
- Stratification matters because fraud is rare; a naive split can distort the class balance a model actually sees during training.

Experimentation is an engineering discipline

- The purpose of experimentation is not to collect many scores, but to support a credible model choice.
- Every run should answer a question: does a new feature help, does a different model family help, or does tuning justify extra complexity?
- Fix the evaluation protocol and decide what counts as a meaningful improvement before looking at results.
- When the question is unclear, experiment tracking becomes a storage system for confusion rather than evidence.

What must be tracked, and why

Dataset version / extraction date

So a result can be reproduced against the same data later.

Feature pipeline version

So preprocessing changes are not confused with model changes.

Hyperparameters and seeds

So the exact configuration that produced a result is known.

Metrics, confusion matrices, artifacts

So the trained output, not just a number, can be audited.

Code commit or notebook snapshot

So the logic behind a run can be replayed months later.

Fair comparison versus common bias

A fair comparison has

- The same dataset split or protocol
- The same preprocessing, unless that is the experiment
- Metrics chosen for the real decision

Bias creeps in from

- Reusing the test set until it becomes part of tuning
- Mixing notebook state and hidden edits
- Reporting only the winning run

Manual tuning	Small problems and pedagogical intuition.
Grid search	Small, well-structured search spaces.
Random search	Better coverage when many hyperparameters matter unevenly.
Bayesian optimization	Expensive models where each run has high cost.

- A tuned model should beat a simple, previously deployed, or default baseline by a meaningful margin, not just a decimal point.

Neptune example: defining a search space

```
model = GradientBoostingRegressor(random_state=42)

param_grid = {
    "n_estimators": [100, 200, 300],
    "max_depth": [3, 5, 7],
    "learning_rate": [0.05, 0.1, 0.2],
    "min_samples_split": [2, 5, 10],
}
```

- From 'notebooks/Class7_Model_Tuning_Neptune.ipynb', which uses the Neptune.ai experiment tracker alongside 'GridSearchCV'.
- The grid gives a realistic trade-off among model complexity, fit time, and predictive gain.

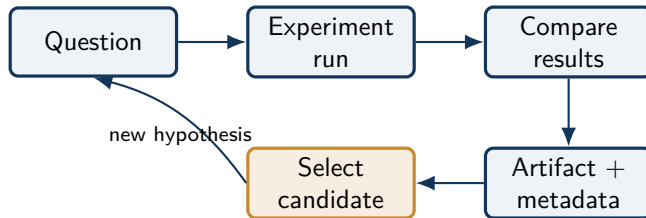
Neptune example: tuning and logging together

```
grid_search.fit(X_train, y_train)

run["best/parameters"] = grid_search.best_params_
run["best/cv_score"] = grid_search.best_score_
```

- The important idea is not Neptune specifically, but the discipline of storing parameters and metrics together, in the same place, automatically.
- That same pattern maps naturally onto MLflow, Weights & Biases, or an internal experiment registry.

From notebook run to governed asset



Neptune example: evaluate and persist the winner

```
best_model = grid_search.best_estimator_  
y_pred = best_model.predict(X_test)  
  
run["test/mse"] = mean_squared_error(y_test, y_pred)  
run["test/r2"] = r2_score(y_test, y_pred)  
  
joblib.dump(best_model, "best_gb_model.pkl")  
run["artifacts/model"].upload("best_gb_model.pkl")
```

- A useful experiment ends with a traceable artifact, not only a screenshot of the best score.
- This is also the natural bridge to model registries and deployment, covered fully in Class 20.

Common tuning mistakes

- Tuning on the test set.
- Comparing runs trained on different preprocessing logic.
- Reporting only the best metric, not its variability or computational cost.
- Forgetting that a slightly weaker model may be much easier to deploy, explain, or monitor.

From a winning run to a real decision

- Tracking tells us which run won. It does not yet tell us whether that run is good for the decision it supports.
- Imbalanced classification needs precision, recall, PR-AUC, and threshold analysis, not accuracy alone.
- Regression needs error-magnitude metrics such as MAE or RMSE, chosen for the unit that matters operationally.
- Calibrated probabilities matter whenever a downstream action depends on confidence, not just the predicted class.
- Operational cost matters as much as any single numerical score.

Fraud dataset reality: extreme class imbalance



From 'Credit_Card_Fraud_Detection_Solution.ipynb': the first modeling challenge is not algorithm choice but the rarity of the positive class.

Repository example: fit, predict, and inspect errors

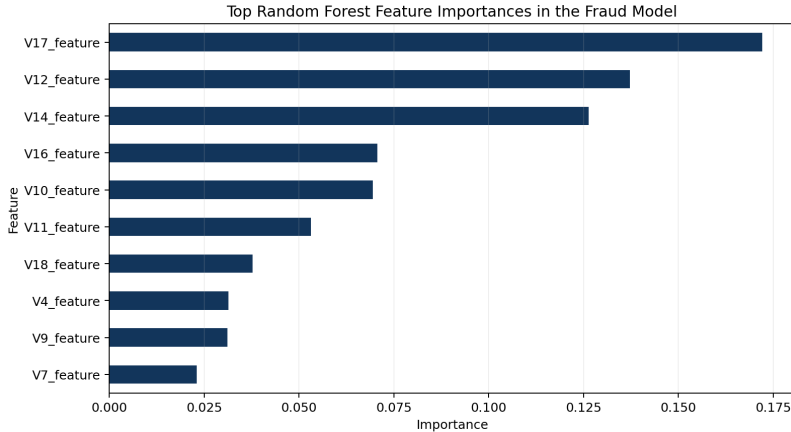
```
y_pred = best_model.predict(X_test_scaled)

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

- From 'Credit_Card_Fraud_Detection_Solution.ipynb': the confusion matrix exposes missed fraud cases directly, which a single accuracy number cannot.
- The classification report opens the door to threshold and cost-sensitive evaluation.

Model interpretation: top feature importances



Generated from the saved 'streamlit/model.pkl': tree-based models can highlight influential variables, but importance is still not the same as causality.

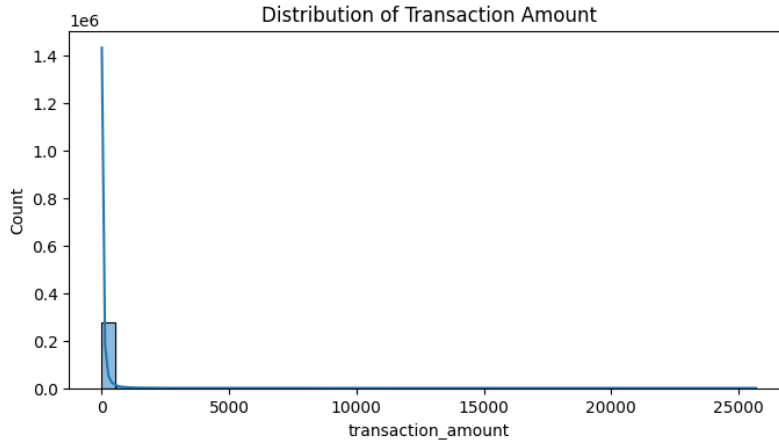
Why accuracy can fail badly

Fraud example

If only 0.2% of transactions are fraudulent, a model that predicts “legitimate” every time may look highly accurate while being operationally useless.

- This is why threshold, recall, and precision are part of the engineering conversation, not an afterthought.

Distribution shape interacts with evaluation



The transaction amount distribution is highly skewed, a reminder that evaluation, scaling, and threshold choice all interact with each other.

A model creates value only through an interface

- A trained model that nobody can call or interpret has limited practical value, however good its offline metrics look.
- An interface makes predictions available to users, services, and operational workflows.
- The interface also shapes trust: input checks, response format, and clarity all matter as much as the metric that selected the model.

Prediction is not the final decision

- A fraud score suggests escalation, not guilt.
- A process alert suggests investigation, not automatic shutdown.
- Interfaces should communicate uncertainty, threshold logic, and next actions whenever possible.

Engineering implication

The threshold that turns a probability into a decision is itself a design choice, not a byproduct of the model.

Repository example: threshold-driven decision support

```
if st.button("Predict Fraud Risk"):
    prediction = model.predict(scaled)[0]
    probability = model.predict_proba(scaled)[0][1]

    if prediction == 1:
        st.error(f"Fraud Probability: {probability:.2%}")
    else:
        st.success(f"Fraud Probability: {probability:.2%}")
```

- From 'streamlit/pages/App_Credit_Card.py': the interface exposes both the decision and the probability behind it.
- A human user needs labels, context, and actionable feedback, not raw class labels or JSON.

What a better dashboard should include

- A visible explanation of what the model does and does not do.
- Controls that reflect meaningful business inputs, not arbitrary technical internals.
- Results that support action: alert, score, trend, or explanation.
- Consistent links to data provenance, model version, or monitoring signals.

Where trust breaks down

- The interface hides uncertainty behind a single confident-looking number.
- The inputs shown to users do not match the assumptions the model was trained under.
- The threshold was chosen once, informally, and never revisited as the data changed.
- The dashboard makes a demo model look authoritative when it has not been validated for that purpose.

Repo activity

- 'notebooks/Class6_Model_Training_.ipynb' gives the baseline split, tune, evaluate, and serialize workflow.
- 'notebooks/Class7_Model_Tuning_Neptune.ipynb' adds Neptune.ai tracking on top of the same grid-search pattern.
- 'Credit_Card_Fraud_Detection_Solution.ipynb' supplies the evaluation workflow this class is grounded in.
- 'streamlit/pages/App_Credit_Card.py' turns the same model into a threshold-driven, human-facing decision.
- All four artifacts share one model and one scaler, so tracking, evaluation, and decision-making must stay consistent with each other.

Question

Two runs report similar cross-validation scores but were trained on differently scaled inputs. Can we trust the comparison?

- This connects tracked metadata directly back to the “what must be tracked” table earlier in this class.

Question

The fraud model has high recall but noticeably lower precision at the deployed threshold. Should we ship it as-is?

- This forces students to separate metric quality from decision quality and operational cost.
- It also motivates the online adaptation and serving discussions in Classes 18 and 20.

Papers and materials

Neptune documentation; MLflow documentation; scikit-learn model selection guide; Bergstra and Bengio (2012), Random Search; Bergstra et al. (2013), Hyperopt; scikit-learn model evaluation guide; Domingos (2012), A Few Useful Things to Know About Machine Learning; Chip Huyen, Designing Machine Learning Systems; Streamlit documentation.

From This Class to the Next

What stays with us

- Model selection is tied to the prediction task, the error costs, and the constraints of the system it will live in.
- A useful experiment records data, configuration, metrics, and artifacts together, automatically.
- Hyperparameter search is valuable only when it is reproducible, comparable, and beats a real baseline.
- Metrics are decision tools: a prediction becomes a decision only once thresholds, uncertainty, and interface design are made explicit.
- Trust in a model interface is earned through validation, clarity, and honest communication of limits.

Project use

Next connection